4BUISO14C Business Computing

Module Leader : Ms Abarnah Kirupananda

Name : Sajini Malinhara Udawela

IIT ID : 20241013

UOW ID : W2149487

Degree : Business Data Analytics

Table of Contents

1. ABSTRACT ____ 3
2. ACKNOWLEDGEMENT ____ 3
3. JUSTIFICATION ____ 4
   3.1 ONLINEDB MODULE ____ 4
   3.2 QUERYDATA MODULE ____ 5
   3.3 VIZDATA MODULE ____ 6
   3.4 MAIN.IPYNB ____ 7
4. INSIGHTS AND RESULTS ____ 8
   4.1TOP SELLING PEODUCTS ____ 8
   4.2 SALES TRENDS BY MONTH ____ 8
   4.3 TOP 5 CUSTOMERS ____ 8
   4.4 REVENUE CONTRIBUTION BY PRODUCT ____ 8
5. APPENDIX : SOURCE CODE ____ 9
   5.1 ONLINEDB.PY ____ 9
   5.2 QUERYDATA.PY ____ 10
   5.3 VIZDATA.PY ____ 12
   5.4 MAIN.IPYNB ____ 14

Figure 1 ____ 17
Figure 2 ____ 17
Figure 3 ____ 18
Figure 4 ____ 18
Figure 5 ____ 18
Figure 6 ____ 19
Figure 7 ____ 19
Figure 8 ____ 20
Figure 9 ____ 20
Figure 10 ____ 21
Figure 11 ____ 22
Figure 12 ____ 22
Figure 13 ____ 22

Table 1 : OnlineDB ____ 4
Table 2 : QueryData ____ 5
Table 3 : VizData ____ 7

1. Abstract

This report aims to design and implement a small-scale business application using python that handles database creation, data manipulkation and data visualisation for an online retail business.

The system is modularised into three key parts:

- **OnlineDB** (Database creation and population)
- **QueryData** (Data analysis using SQL queries)
- **VizData** (Data visualisation

2. Acknowledgement

I would like to express my gratitude to Ms Abarnah Kirupananda for her astute leadership, Which enabled me to successfully complete this coursework. Thank you for being there for me every time I needed it.

3. Justification

3.1 OnlineDB Module

Table 1 : OnlineDB

Function	Explanation

`import pandas as pd`
 	
`import sqlite3 as sq3`
 	
`import os`
	By calling pandas, it lets you work with dataframes.
	
	By calling sqlite, it lets you connect your code to a SQLite database.
	
	By importing os, it works with file paths. Like joins folder and file names safely.

`create_sqlite_connection`	Connects to a SQLite database at the given path. If the databse doesn't exist it will create it automatically.
`conn.execute("PRAGMA foreign_keys = ON;")`	Turns on "foreign key constraints" which are oP by defult in SQLit. So this ensure data consistency between tables.
`return conn`	Returns to connection object.
`create_tables(conn):`	Creates 3 tables : Customers, Products, Transactions.
`cursor = conn.cursor()` `cursor.execute(`	Creates a curson which executes SQL commands.
`dataframe(folder, filename):`	Loads the CSV files into pandas dataframes using specified folder and filename.
`inserting_df__db(df, table_name,conn):`	Inserts a cleaned DataFrame into a specified database

`If_exists = 'replace'` `Index = False`	If the table already exists, it deletes and replaces it Do not insert the pandas index into the databse

`clean_df(df):`
 	
`df = df.dropna()`
 	
`df = df.drop_duplicates()`
	Removes missing and duplicate records from the dataframe.
	
	Removes any rows with missing data (dropna)
	
	Removes duplicate rows (drop_duplicates)

3.2 QueryData Module

Table 2 : QueryData

Function	Explanation
`running_query(query_string, conn):`	Executes any SQL query and returns results as a DataFrame.
`total_quantity_per_product_monthly(conn):`	Calculates total quantity sold per product per month. Grouping by both StockCode and month.
`sales_trends_per_product(conn):`	Calculates total monthly sales per product per month. Joins Transactions with Products to get product descriptions.
`top_5_customers_by_sales(conn):`	Retrieves top 5 customers by total sales. Used **`LIMIT 5`** to return only top 5 .
`revenue_by_product(conn):`	Computes total revenue per product. By using **`DESC`** it shows the products ordered from most profitable products to least profitsble

3.3 VizData Module

Function	Explanation
`plot_total_quantity_bar(conn):`	To Get a bar plot of top products by quantitiy sold.
`tot_quan_bar_df = df.sort_values(by='TotalQuantity', ascending = False)`	Sorts by quantity and sorts the values in descending (from highest to lowest) order
`sns.barplot(data=df.head(11), x='StockCode', y='TotalQuantity', errorbar = None)`	Plots the top 12 products .

`plt.xlabel("Product")`
`plt.ylabel("Total Quantity")`
`plt.xticks(rotation=45)`
	Adding X axis and Y axis name also since theres a bit more data to stop overlaping the labels [`plt.xticks(rotation=45)`] helps by making the labes angular.

`plot_sales_trends_line(conn):`	A line chart showing monthly sales trends per product.
`top_products = df.groupby ('Description')['MonthlySales'] .sum().nlargest(10).index`	Groups the data by Description. Sums MonthlySales for each product and selects the top 10 products with highest total sales.
`top5_df = df[df['Description'] .isin(top_products)]`	Keeps only the rows where the product is in that top 10 list.
`hue = 'Description'`	To split lines by product.
`plot_top_customers_barh(conn):`	Horizontal bar chart for top 5 customers (Total sales per customer) .
`.astype(str)`	This ensures CustomerID values are treated as text and avoids misinterpretation.

`plot_revenue_pie_chart(conn):`	A pie chart of top 5 products and others by revenue.
`other_row = pd.DataFrame([{'Description': 'Other', 'Revenue': others_total}])`	Sums the remaining products into “ Other ”. concatenates into a single dataframe and plot a pie chart.
`autopct='%1.1f%%',`	To show clean percentages.

Table 3 : VizData

3.4 Main.ipynb

This Jupyer Notebook acts as the main entry point for executing the full application pipeline. It imports and integrates all previously defined modules : OnlineDB, QueryData, and VizData.

Functionality

Database Setup : Calls functions from OnlineDB to create the SQLite database and populate it with cleaned data.

Query Execution : Runs business-related queries via QueryData to retrive important sales and customer insights .

Data Visualisation : Uses VizData to produce clear and informative graphs based on query results .

4. Insights and Results

4.1Top Selling Products

- As I see in this bar chart, it shows that a small set of products dominate sales volume. These products should be prioritized for stock management and marketing.

4.2 Sales Trends by Month

- As in my view of this line chart, it shows that product sales vary significantly by month. Some products shows a spike during specific periods, which is indicating seasonal demand. This insight can help time promotions or restocking ePorts more ePectively.

4.3 Top 5 Customers

- The horizontal bar chart shows the top 5 customers contributed significantly to the overall sales revenue. Focusing on these, high-value customers through loyalty programs or exclusive oPers could increase revenue futher .

4.4 Revenue Contribution by Product

- According to the pie chart, it shows that a small percentage of products generate the majority of revenue. Business decisions should focus on maintaining and expanding these top-performing products while evaluating whether low-revenue products are worth continuing.

5. Appendix : Source code

5.1 OnlineDB.py

```
import pandas as pd  import sqlite3 as sq3 import os  def
create_sqlite_connection(location_path, db_name):
full_path = os.path.join(location_path, db_name)     conn
= sq3.connect(full_path)     conn.execute("PRAGMA
foreign_keys = ON;")     print(f"Database created
successfully at {full_path}")     return conn def
create_tables(conn):     cursor = conn.cursor()
cursor.execute('''
        CREATE TABLE IF NOT EXISTS Customers (
            CustomerID INTEGER PRIMARY KEY,
            Country TEXT NOT NULL
        );
    ''')
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS Products (
            StockCode TEXT PRIMARY KEY,
            Description TEXT NOT NULL,
            UnitPrice REAL NOT NULL
        );     ''')
cursor.execute('''
CREATE TABLE IF NOT
EXISTS Transactions (

            TransactionID INTEGER PRIMARY KEY AUTOINCREMENT,
            InvoiceNo TEXT NOT NULL,
            CustomerID INTEGER,
            StockCode TEXT,
            Quantity INTEGER NOT NULL,
```

```
            InvoiceDate TEXT NOT NULL,
            UnitPrice REAL,
            TotalAmount_GBP REAL NOT NULL,
            FOREIGN KEY(CustomerID) REFERENCES
Customers(CustomerID),
            FOREIGN KEY(StockCode) REFERENCES Products(StockCode)
        );
''')
    print('Tables created successfully')
      import os  def dataframe(folder,
filename):     locatefile =
os.path.join(folder, filename)     df =
pd.read_csv(locatefile)     return df
 def clean_df(df):     df =
df.dropna()     df =
df.drop_duplicates()
return df
  def inserting_df__db(df,
table_name,conn):
df.to_sql(table_name,conn, if_exists =
'replace', index= False)
print(f'{table_name} table populated
successfully.')

conn.commit()
conn.close()
```

5.2 QueryData.py

```
import  pandas  as  pd
import sqlite3 as sq3
```

```
 def       running_query(query_string,
conn):
    return pd.read_sql_query(query_string, conn)
 def
total_quantity_per_product_monthly(conn):


    query = '''
    SELECT StockCode, strftime('%Y-%m', InvoiceDate) AS Month,
SUM(Quantity) AS TotalQuantity
    FROM Transactions
    GROUP   BY   StockCode,   Month
ORDER BY Month, StockCode;
    '''
    return running_query(query, conn)
 def
sales_trends_per_product(conn):
         query
= '''
    SELECT p.Description, strftime('%Y-%m', t.InvoiceDate) AS
Month, SUM(t.TotalAmount_GBP) AS MonthlySales
    FROM Transactions t      JOIN Products p ON
t.StockCode = p.StockCode
    GROUP   BY   p.Description,   Month
ORDER BY Month, p.Description;
    '''
    return running_query(query, conn)
 def
top_5_customers_by_sales(conn):
    query = '''
    SELECT c.CustomerID, SUM(t.TotalAmount_GBP) AS TotalSales
```

```
    FROM Transactions t
    JOIN Customers c ON t.CustomerID = c.CustomerID
    GROUP    BY    c.CustomerID
ORDER BY TotalSales DESC
    LIMIT     5;
'''
    return running_query(query, conn)
 def
revenue_by_product(conn):
    query = '''
    SELECT p.Description, SUM(t.TotalAmount_GBP) AS
TotalRevenue
    FROM Transactions t      JOIN Products p ON
t.StockCode = p.StockCode
    GROUP    BY    p.Description
ORDER BY TotalRevenue DESC;
    '''
    return running_query(query, conn)
```

5.3 VizData.py

```
import sqlite3 as sq3 import
pandas as pd import
matplotlib.pyplot as plt import
seaborn as sns  import
QueryData as qd
sns.set(style="whitegrid")
 def
plot_total_quantity_bar(conn):

    df = qd.total_quantity_per_product_monthly(conn)
```

```
    tot_quan_bar_df = df.sort_values(by='TotalQuantity', ascending =
False)

    sns.barplot(data=df.head(12), x='StockCode', y='TotalQuantity',
errorbar = None)

    plt.title("Top 12 products by quantity sold")

plt.xlabel("Product")     plt.ylabel("Total

Quantity")     plt.xticks(rotation=45)

plt.tight_layout()     plt.show()

 def

plot_sales_trends_line(conn):


    df = qd.sales_trends_per_product(conn)

top_products =

df.groupby('Description')['MonthlySales'].sum().nlargest(10).index

top5_df = df[df['Description'].isin(top_products)]

    sns.lineplot(x='Month', y='MonthlySales', hue = 'Description'
,data=top5_df, marker = "o")

    plt.title("Monthly sales trends per product")

plt.xlabel("Month")     plt.ylabel("Total Sales")

plt.xticks(rotation=45)      plt.tight_layout()

plt.show()

 def

plot_top_customers_barh(conn):


    df = qd.top_5_customers_by_sales(conn)

df['CustomerID'] = df['CustomerID'].astype(str)

sns.barplot(x='TotalSales', y='CustomerID', data=df)

plt.title("Top 5 customers by total sales")

plt.xlabel("Total Sales")     plt.ylabel("Customer ID")

plt.tight_layout()     plt.show()

 def plot_revenue_pie_chart(conn):

df = qd.revenue_by_product(conn)
```

```
    top_df = df.head(5).copy()
    top_df = top_df.rename(columns={'TotalRevenue': 'Revenue'})
others_total = df['TotalRevenue'].iloc[5:].sum()     other_row =
pd.DataFrame([{'Description': 'Other', 'Revenue': others_total}])

    pie_df = pd.concat([top_df, other_row], ignore_index=True)

    plt.pie(pie_df['Revenue'], labels=pie_df['Description'],
autopct='%1.1f%%', startangle=180)
    plt.title("Revenue contribution of each product")
plt.tight_layout()     plt.show()
```

5.4 Main.ipynb

```
import OnlineDB as odb
import sqlite3 as sq3
import pandas as pd import
QueryData as qd import
VizData as vz

%pwd
location_path = "/Users/sajiniudawela/Desktop/OnlineSales" db_name
= "OnlineDB"

conn = odb.create_sqlite_connection(location_path, db_name)

odb.create_tables(conn)

loca = '/Users/sajiniudawela/Desktop/OnlineSales'
filename = 'customers.csv' cust_df =
odb.dataframe(loca, filename) cust_df.head()

customers_df = odb.clean_df(cust_df)
customers_df.head
```

```
conn = sq3.connect('OnlineDB.db')

odb.inserting_df__db(cust_df, 'Customers', conn)


loca = '/Users/sajiniudawela/Desktop/OnlineSales'

filename = 'products.csv'

products_df = odb.dataframe(loca, filename)

products_df.head()


products_df = odb.clean_df(products_df)
products_df.head


conn = sq3.connect('OnlineDB.db')

odb.inserting_df__db(products_df, 'Products', conn)


loca = '/Users/sajiniudawela/Desktop/OnlineSales'

filename = 'transactions.csv'

transactions_df = odb.dataframe(loca, filename)

transactions_df.head()


transactions_df = odb.clean_df(transactions_df)

transactions_df.head


conn = sq3.connect('OnlineDB.db')


cust_df.to_sql('Customers', conn, if_exists='replace', index=False)

products_df.to_sql('Products', conn, if_exists='replace', index=False)

transactions_df.to_sql('Transactions', conn, if_exists='replace',

index=False)


conn = sq3.connect('OnlineDB.db')
```

```
df = qd.total_quantity_per_product_monthly(conn)

df.head()


conn = sq3.connect('OnlineDB.db') df =

qd.sales_trends_per_product(conn)

df.head()


conn = sq3.connect('OnlineDB.db')
df = qd.top_5_customers_by_sales(conn)

df.head()

conn = sq3.connect('OnlineDB.db')

df = qd.revenue_by_product(conn)

df.head()

vz.plot_total_quantity_bar(conn)


vz.plot_sales_trends_line(conn)


vz.plot_top_customers_barh(conn)


vz.plot_revenue_pie_chart(conn)
```

```
[1]: import OnlineDB as odb
     import sqlite3 as sq3
     import pandas as pd
     import QueryData as qd
     import VizData as vz

[2]: %pwd

[2]: '/Users/sajiniudawela/OnlineSales'

[3]: location_path = "/Users/sajiniudawela/Desktop/OnlineSales"
     db_name = "OnlineDB"

     conn = odb.create_sqlite_connection(location_path, db_name)

     Database created successfully at /Users/sajiniudawela/Desktop/OnlineSalesOnlineDB

[4]: odb.create_tables(conn)

     Tables created successfully
```

Figure 1

```
[5]: loca = '/Users/sajiniudawela/Desktop/OnlineSales'
     filename = 'customers.csv'
     cust_df = odb.dataframe(loca, filename)
     cust_df.head()
```

[5]:

	CustomerID	Country
0	13085.0	United Kingdom
1	13078.0	United Kingdom
2	15362.0	United Kingdom
3	18102.0	United Kingdom
4	12682.0	France

```
[6]: customers_df = odb.clean_df(cust_df)
     customers_df.head
```

```
[6]: <bound method NDFrame.head of       CustomerID         Country
     0        13085.0  United Kingdom
     1        13078.0  United Kingdom
     2        15362.0  United Kingdom
     3        18102.0  United Kingdom
     4        12682.0          France
     ...          ...             ...
     4396     18269.0  United Kingdom
     4397     13270.0  United Kingdom
     4398     12942.0  United Kingdom
     4399     13369.0  United Kingdom
     4400     15211.0  United Kingdom

     [4388 rows x 2 columns]>
```

Figure 2

```
[7]: conn = sq3.connect('OnlineDB.db')
     odb.inserting_df__db(cust_df, 'Customers', conn)
```

```
Customers table populated successfully.
```

```
[8]: loca = '/Users/sajiniudawela/Desktop/OnlineSales'
     filename = 'products.csv'
     products_df = odb.dataframe(loca, filename)
     products_df.head()
```

[8]:

	StockCode	Description	UnitPrice
0	85048	15CM CHRISTMAS GLASS BALL 20 LIGHTS	6.95
1	79323P	PINK CHERRY LIGHTS	6.75
2	79323W	WHITE CHERRY LIGHTS	6.75
3	22041	RECORD FRAME 7" SINGLE SIZE	2.10
4	21232	STRAWBERRY CERAMIC TRINKET BOX	1.25

Figure 3

```
[9]: products_df = odb.clean_df(products_df)
     products_df.head

[9]: <bound method NDFrame.head of       StockCode                         Description  Un
     itPrice
     0         85048  15CM CHRISTMAS GLASS BALL 20 LIGHTS       6.95
     1        79323P                   PINK CHERRY LIGHTS       6.75
     2        79323W                  WHITE CHERRY LIGHTS       6.75
     3         22041         RECORD FRAME 7" SINGLE SIZE        2.10
     4         21232       STRAWBERRY CERAMIC TRINKET BOX       1.25
     ...         ...                                 ...        ...
     19022       DOT                       DOTCOM POSTAGE     822.94
     19023     22838        3 TIER CAKE TIN RED AND CREAM      29.95
     19024       DOT                       DOTCOM POSTAGE      85.79
     19028    46000S                         Dotcom sales       0.00
     19029    46000M                         Dotcom sales       0.00

     [17110 rows x 3 columns]>

[10]: conn = sq3.connect('OnlineDB.db')
      odb.inserting_df__db(products_df, 'Products', conn)

      Products table populated successfully.
```

Figure 4

```
[11]: loca = '/Users/sajiniudawela/Desktop/OnlineSales'
      filename = 'transactions.csv'
      transactions_df = odb.dataframe(loca, filename)
      transactions_df.head()
```

[11]:

	InvoiceNo	CustomerID	StockCode	Quantity	InvoiceDate	UnitPrice	TotalAmount_GBP
0	489434	13085.0	85048	12	2009-12-01 07:45:00	6.95	83.4
1	489434	13085.0	79323P	12	2009-12-01 07:45:00	6.75	81.0
2	489434	13085.0	79323W	12	2009-12-01 07:45:00	6.75	81.0
3	489434	13085.0	22041	48	2009-12-01 07:45:00	2.10	100.8
4	489434	13085.0	21232	24	2009-12-01 07:45:00	1.25	30.0

Figure 5

```
[12]: transactions_df = odb.clean_df(transactions_df)
      transactions_df.head

[12]: <bound method NDFrame.head of        InvoiceNo  CustomerID StockCode  Quantity
      InvoiceDate  \
      0         489434     13085.0     85048        12  2009-12-01 07:45:00
      1         489434     13085.0    79323P        12  2009-12-01 07:45:00
      2         489434     13085.0    79323W        12  2009-12-01 07:45:00
      3         489434     13085.0     22041        48  2009-12-01 07:45:00
      4         489434     13085.0     21232        24  2009-12-01 07:45:00
      ...          ...         ...       ...       ...                  ...
      525456    538171     17530.0     22271         2  2010-12-09 20:01:00
      525457    538171     17530.0     22750         1  2010-12-09 20:01:00
      525458    538171     17530.0     22751         1  2010-12-09 20:01:00
      525459    538171     17530.0     20970         2  2010-12-09 20:01:00
      525460    538171     17530.0     21931         2  2010-12-09 20:01:00

              UnitPrice  TotalAmount_GBP
      0            6.95            83.40
      1            6.75            81.00
      2            6.75            81.00
      3            2.10           100.80
      4            1.25            30.00
      ...           ...              ...
      525456       2.95             5.90
      525457       3.75             3.75
      525458       3.75             3.75
      525459       3.75             7.50
      525460       1.95             3.90

      [410763 rows x 7 columns]>
```

Figure 6

```
[13]: conn = sq3.connect('OnlineDB.db')

[14]: cust_df.to_sql('Customers', conn, if_exists='replace', index=False)
      products_df.to_sql('Products', conn, if_exists='replace', index=False)
      transactions_df.to_sql('Transactions', conn, if_exists='replace', index=False)

[14]: 410763

[15]: conn = sq3.connect('OnlineDB.db')
      df = qd.total_quantity_per_product_monthly(conn)
      df.head()
```

[15]:

	StockCode	Month	TotalQuantity
0	10135	2009-00	10
1	15056BL	2009-00	3
2	15056N	2009-00	11
3	16011	2009-00	48
4	16161U	2009-00	25

Figure 7

```
[16]: conn = sq3.connect('OnlineDB.db')
      df = qd.sales_trends_per_product(conn)
      df.head()
```

[16]:

	Description	Month	MonthlySales
0	DOORMAT UNION JACK GUNS AND ROSES	2009-12	5431.95
1	3 STRIPEY MICE FELTCRAFT	2009-12	1932.60
2	4 PURPLE FLOCK DINNER CANDLES	2009-12	643.14
3	ANIMAL STICKERS	2009-12	45.36
4	BLACK PIRATE TREASURE CHEST	2009-12	193.05

```
[17]: conn = sq3.connect('OnlineDB.db')
      df = qd.top_5_customers_by_sales(conn)
      df.head()
```

[17]:

	CustomerID	TotalSales
0	18102.0	341776.73
1	14646.0	243853.05
2	14156.0	183163.55
3	14911.0	137710.76
4	13694.0	128172.42

Figure 8

```
[18]: conn = sq3.connect('OnlineDB.db')
      df = qd.revenue_by_product(conn)
      df.head()
```

[18]:

	Description	TotalRevenue
0	POSTAGE	4142398.26
1	WHITE HANGING HEART T-LIGHT HOLDER	1931689.63
2	REGENCY CAKESTAND 3 TIER	1230304.95
3	ASSORTED COLOUR BIRD ORNAMENT	626869.44
4	JUMBO BAG RED RETROSPOT	510390.90

Figure 9

```
[19]: vz.plot_total_quantity_bar(conn)
```

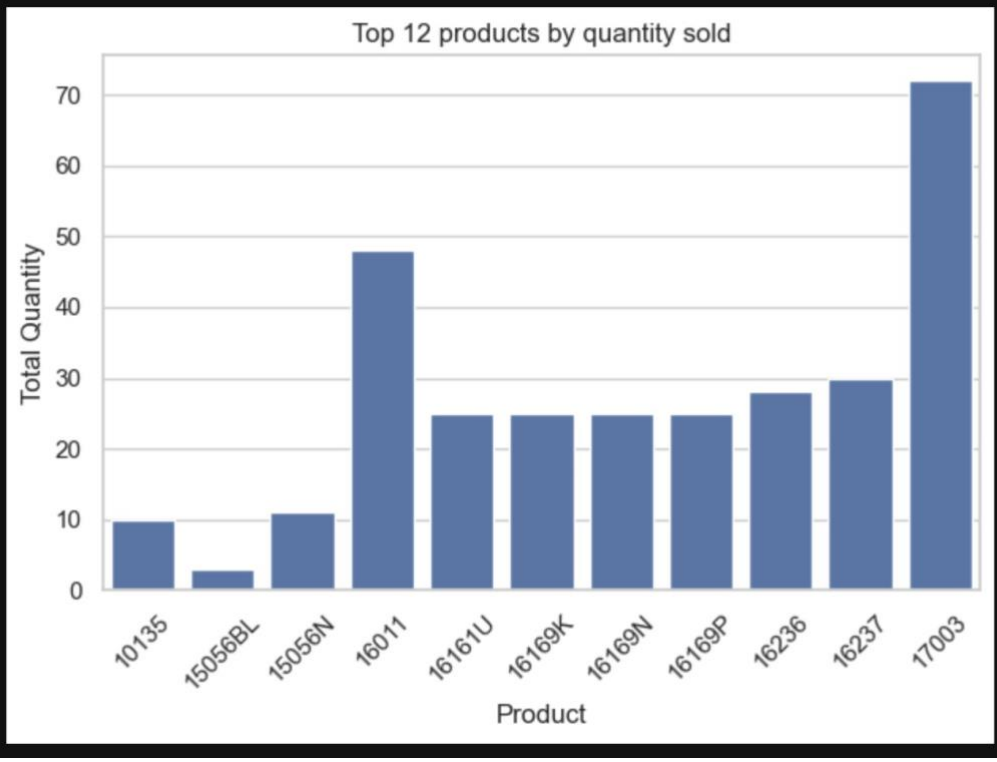


Figure 10

```
[20]: vz.plot_sales_trends_line(conn)
```

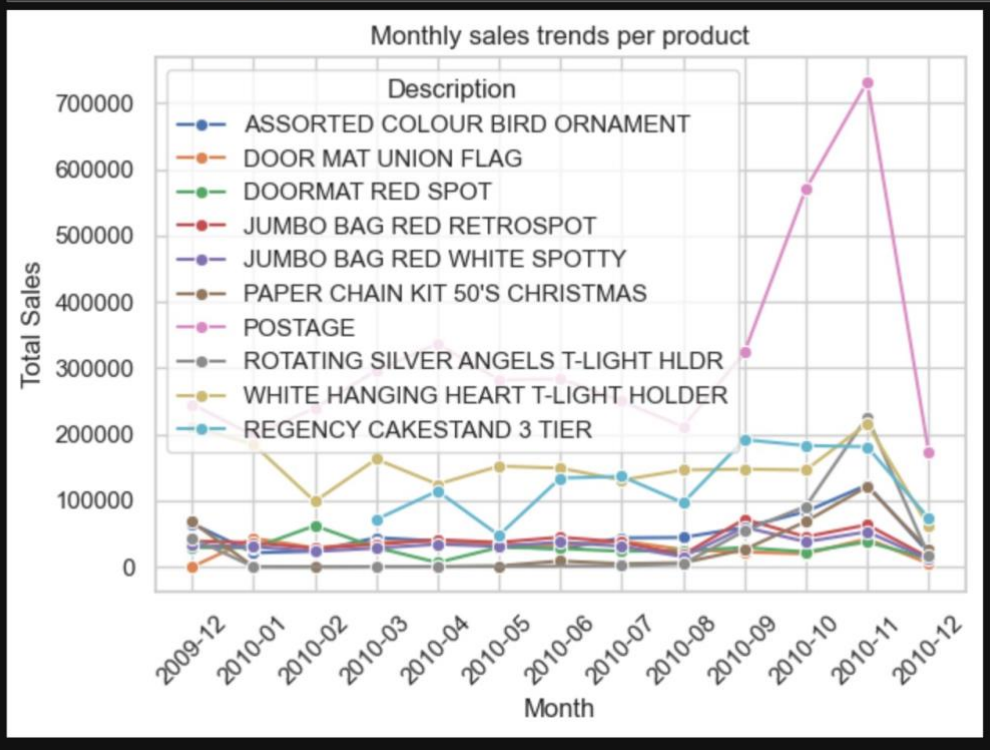


Figure 11

```
[21]: vz.plot_top_customers_barh(conn)
```

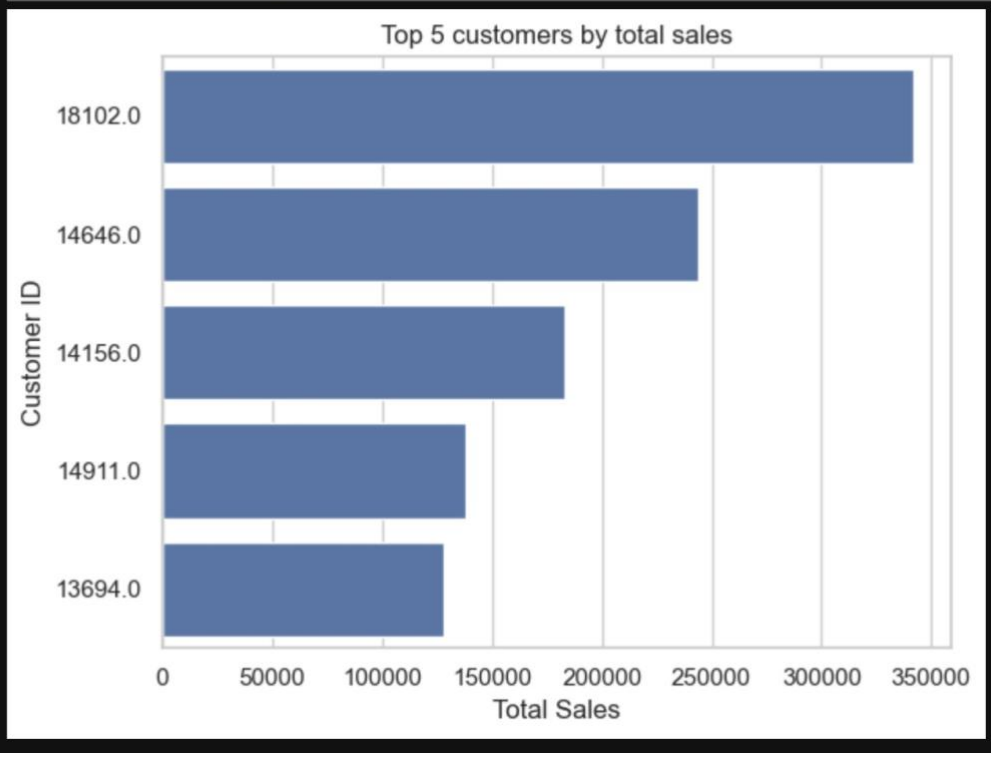
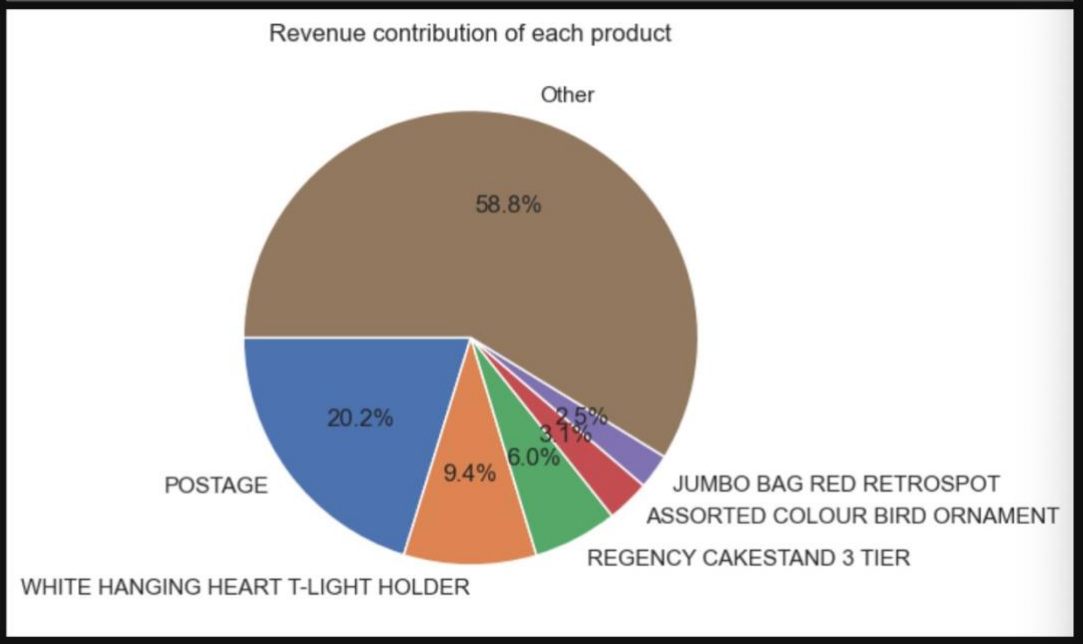


Figure 12

```
[23]: vz.plot_revenue_pie_chart(conn)
```




```
[ ]:
```

Figure 13